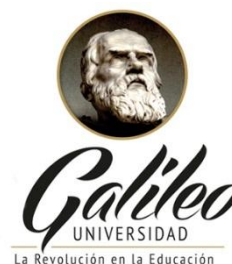


|                                  |                              |
|----------------------------------|------------------------------|
| Universidad Galileo              | Fecha 4/10/2026              |
| Facultad: FISSIC                 | Alumno: Pablo Urbina         |
| Curso: Microprocesadores         | Carnet: 24001508             |
| Sección: A                       | Hora de Laboratorio: 1:00 PM |
| Auxiliar:                        | Día de Laboratorio: Lunes    |
| Brandon Eduardo Mendoza Espinoza |                              |



## Laboratorio #6 Puertos Paralelos (GPIO)

### Objetivos:

Aprender a controlar los puertos paralelos de una tarjeta utilizando VS Code y Keil MDK6.

### Resumen:

Se construyo un circuito con pulsadores en configuración pull down para encender LEDs simulando un joystick, donde el circuito se controlo principalmente por medio de el microcontrolador utilizando ciertos pines, los cuales se configuraron programando en C las configuraciones de entrada y salida, pull-down y escuchar las interrupciones de cuando se presiona un botón utilizando polling para prender el LED respectivo.

### Teoría:

Los puertos GPIO son puertos de entrada y salida de propósito general, los cuales permiten que un microcontrolador intercambie señales con dispositivos externos. Cada pin puede configurarse independientemente como entrada digital, salida digital, función alternativa o conexión analógica de acuerdo con lo que se necesite, permitiendo el uso de un mismo microcontrolador en distintas aplicaciones. La cantidad de pines depende de la versión del microcontrolador, en nuestro caso son 16 pines en total. El programa accede a estos periféricos mediante registros, de esta forma, expresiones como GPIOA -> MODER permiten seleccionar un registro del puerto A trabajando desde el lenguaje de C.

La función de cada pin se establece mediante GPIOx\_MODER, el cual utiliza 2 bits por pin de la siguiente forma:

- 00 Entrada digital
- 01 Salida digital
- 10 Función alternativa
- 11 Modo analógico

Para que la configuración sea correcta, se deben modificar ambos bits del campo, los cuales ocupan las posiciones  $2n$  y  $2n+1$ . Al configurar un pin como función alternativa, este queda asociado a un periférico como una interfaz de comunicación o un temporizador, la función específica se escoge mediante los registros `GPIOx_AFRL` y `GPIOx_AFRH`, que destinan 4 bits a cada pin.

La lectura y escritura se hacen mediante registros diferentes. `GPIOx_IDR` permite consultar el nivel lógico detectado en los pines, `GPIOx_ODR` almacena los valores de salida, se puede utilizar una máscara para consultar o modificar únicamente el bit de interés. También está `GPIOx_BSRR`, que permite colocar salidas en 1 o 0 mediante una escritura, donde los bits inferiores seleccionan las salidas que se activan y los bits superiores las salidas que se desactivan.

Cuando una entrada se conecta a un pulsador, se necesita un nivel definido cuando el contacto está abierto. Una resistencia pull-up mantiene la entrada en un nivel alto, mientras que en pull-down la mantiene en un nivel bajo, según la conexión del pulsador, al presionarlo se obtiene el nivel contrario. Es posible seleccionar resistencias internas en el microcontrolador mediante `GPIOx_PUPDR`, cada pin tiene 2 bits:

- 00 Deshabilita ambas resistencias
- 01 Pull-up
- 10 Pull-down

Esta configuración debe corresponder a la manera en la que está construido el circuito para que funcione correctamente.

Antes de utilizar un puerto, se debe habilitar su señal de reloj mediante el módulo RCC, el cual se encarga del control del reloj y reinicio, estas señales se encuentran apagadas de forma predeterminada, por lo que se deben activar con `RCC -> AHB2ENR`. Estos nombres se relacionan con AMBA, que es la arquitectura de interconexión desarrollada por Arm. Dentro de ella, AHB se utiliza para conexiones de alto desempeño y APB para accesos a periféricos que requieren menor ancho de banda.

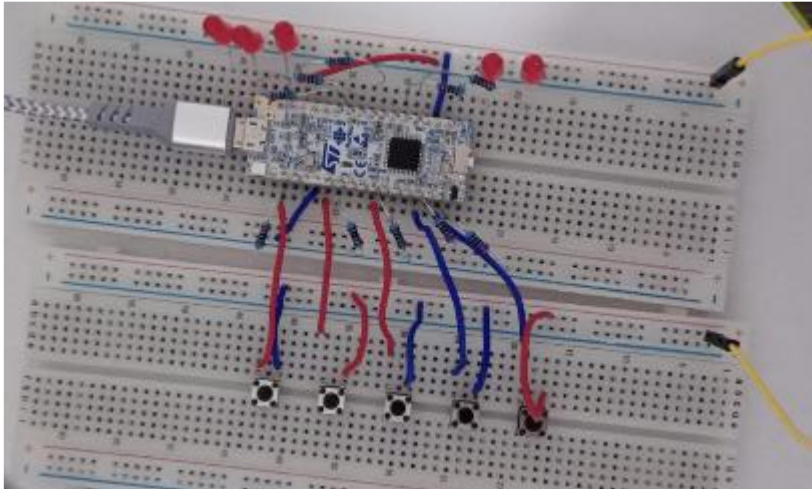
En C, es posible trabajar con máscaras para facilitar el manejo individual de los bits. La definición `#define MASK(x) (1<<x)` crea un macro con parámetro que desplaza el valor 1  $x$  posiciones hacia la izquierda, aunque su uso se parece a una función, el preprocesador sustituye a la macro por su expresión antes de la compilación.

## Datos Prácticos:

Video del circuito funcionando:

[https://drive.google.com/file/d/1lp\\_QDromNiuVeDObPXYAtKO-60k3KLcN/view?usp=sharing](https://drive.google.com/file/d/1lp_QDromNiuVeDObPXYAtKO-60k3KLcN/view?usp=sharing)

Foto del circuito:



## Código

Configuración de los pines utilizados y declaración de funciones

```
// Mascara
#define MASK(x) (1 << (x))

// Pines de entrada (Joystick)
#define PIN_U 9
#define PIN_D 12
#define PIN_L 6
#define PIN_R 11
#define PIN_S 4

// Pines de salida (LEDs)
#define PIN_LD1 7
#define PIN_LD2 6
#define PIN_LD3 5
#define PIN_LD4 1
#define PIN_LD7 0

void habilitar_clock(); // Habilitar la señal de reloj de los GPIO A B

void config_joystick(); // Configurar los pines de JOYSTICK como entrada (MODER = 00) con pull-down (PUPDR = 10)

void config_LEDs(); // Configurar los pines LEDs como salida (MODER = 01)

// Leer inputs del joystick
int up();
int down();
int left();
int right();
int selection();

// Control de LEDs
void encender(int pin);
void apagar_led(int pin);

void apagar();
```

Funciones habilitando el reloj de los GPIO, configurando los pines de entrada y salida en configuración pull-down.

```
void habilitar_clock(){
    RCC -> AHB2ENR |= RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOBEN;
}

void config_joystick(){
    // Configurar los pines de entrada
    GPIOA -> MODER &= ~(3 << (2 * PIN_U));
    GPIOA -> MODER &= ~(3 << (2 * PIN_D));
    GPIOB -> MODER &= ~(3 << (2 * PIN_L));
    GPIOA -> MODER &= ~(3 << (2 * PIN_R));
    GPIOB -> MODER &= ~(3 << (2 * PIN_S));

    // Configurar pull-down, primero se limpia PUPDR y despues se pone en pull-down
    GPIOA -> PUPDR &= ~(3 << (2 * PIN_U));
    GPIOA -> PUPDR |= (2 << (2 * PIN_U));

    GPIOA -> PUPDR &= ~(3 << (2 * PIN_D));
    GPIOA -> PUPDR |= (2 << (2 * PIN_D));

    GPIOB -> PUPDR &= ~(3 << (2 * PIN_L));
    GPIOB -> PUPDR |= (2 << (2 * PIN_L));

    GPIOA -> PUPDR &= ~(3 << (2 * PIN_R));
    GPIOA -> PUPDR |= (2 << (2 * PIN_R));

    GPIOB -> PUPDR &= ~(3 << (2 * PIN_S));
    GPIOB -> PUPDR |= (2 << (2 * PIN_S));
}

void config_LEDs(){
    // Configurar los pines de salida, primero se limpia y despues se pone en 01
    GPIOA -> MODER &= ~(3 << (2 * PIN_LD1));
    GPIOA -> MODER |= (1 << (2 * PIN_LD1));

    GPIOA -> MODER &= ~(3 << (2 * PIN_LD2));
    GPIOA -> MODER |= (1 << (2 * PIN_LD2));

    GPIOA -> MODER &= ~(3 << (2 * PIN_LD3));
    GPIOA -> MODER |= (1 << (2 * PIN_LD3));

    GPIOA -> MODER &= ~(3 << (2 * PIN_LD4));
    GPIOA -> MODER |= (1 << (2 * PIN_LD4));

    GPIOA -> MODER &= ~(3 << (2 * PIN_LD7));
    GPIOA -> MODER |= (1 << (2 * PIN_LD7));
}
```

Detectar que pulsador se presionó y lógica para encender y apagar los LEDs:

```
int up(){
    if((GPIOA -> IDA & MASK(PIN_U))){
        return 1;
    }
    return 0;
}

int down(){
    if((GPIOA -> IDA & MASK(PIN_D))){
        return 1;
    }
    return 0;
}

int left(){
    if((GPIOB -> IDA & MASK(PIN_L))){
        return 1;
    }
    return 0;
}

int right(){
    if((GPIOA -> IDA & MASK(PIN_R))){
        return 1;
    }
    return 0;
}

int selection(){
    if((GPIOB -> IDA & MASK(PIN_S))){
        return 1;
    }
    return 0;
}
```

```
// Encender y apagar los LEDs

void encender(int pin){
    GPIOA -> ODR |= (MASK(pin));
}

void apagar_led(int pin){
    GPIOA -> ODR &= ~(MASK(pin));
}

void apagar(){
    apagar_led(pin: PIN_LD1);
    apagar_led(pin: PIN_LD2);
    apagar_led(pin: PIN_LD3);
    apagar_led(pin: PIN_LD4);
    apagar_led(pin: PIN_LD7);
}
```

## Cálculos Teóricos:

Calcular la resistencia del LED con ley de ohm:

$$V_{fuente} = 3.3 \text{ V} \quad V_{LEDRojo} = 2 \text{ V} \quad I_{LED} = 15 \text{ mA}$$

$$R = \frac{V_{fuente} - V_{LEDRojo}}{I_{LED}} = \frac{3.3 \text{ V} - 2.0 \text{ V}}{15 \text{ mA}} = 86.7 \Omega$$

El cálculo indica que, para no sobrepasar los 15 mA, es necesario utilizar una resistencia mínima de 86.7  $\Omega$ , en la practica se utilizaron resistencias de 220  $\Omega$ .

Calcular corriente teórica que experimento el LED con ley de ohm

$$I_T = \frac{3.3 \text{ V} - 2.0 \text{ V}}{220 \Omega} = 5.91 \text{ mA}$$

Esta corriente es más que segura para que los LEDs enciendan visiblemente y ningún componente del circuito se dañe.

## Conclusiones:

Saber utilizar y configurar los pines de la tarjeta es increíblemente valioso para tener éxito al momento de trabajar con circuitos, la gran variedad de opciones de cada pin, tanto entrada como salida, añade muchísima flexibilidad a la cantidad de cosas que se pueden realizar, aunque hay que ser cauteloso al momento de trabajar con un microcontrolador en un circuito, ya que conectar algo de forma incorrecta o incluso escribir código sin cuidado pueden dañar a la tarjeta, así que hay que ser consciente de lo que se está haciendo y del cómo se está haciendo tanto en la parte de hardware como la parte de software, sabiendo que hacer algo de forma descuidada podría tener consecuencias y echar a perder la tarjeta u otro componente del circuito.

## Bibliografía:

[1] STMicroelectronics, "Getting started with GPIO," *STM32 MCU Wiki*. [En línea]. Disponible en: [https://wiki.st.com/stm32mcu/wiki/Getting\\_started\\_with\\_GPIO](https://wiki.st.com/stm32mcu/wiki/Getting_started_with_GPIO)

[2] Arm, "AMBA." [En línea]. Disponible en: <https://www.arm.com/architecture/system-architectures/amba>

[3] Free Software Foundation, "Function-like Macros," *The C Preprocessor*. [En línea]. Disponible en: <https://gcc.gnu.org/onlinedocs/cpp/Function-like-Macros.html>